TRƯỜNG ĐẠI HỌC
SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
HCMC University of Technology and Education

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ THỐNG THÔNG TIN

GIỚI THIỆU PHÂN TÍCH DỮ LIỆU VÀ NUMPY

GV. P.T.Thể

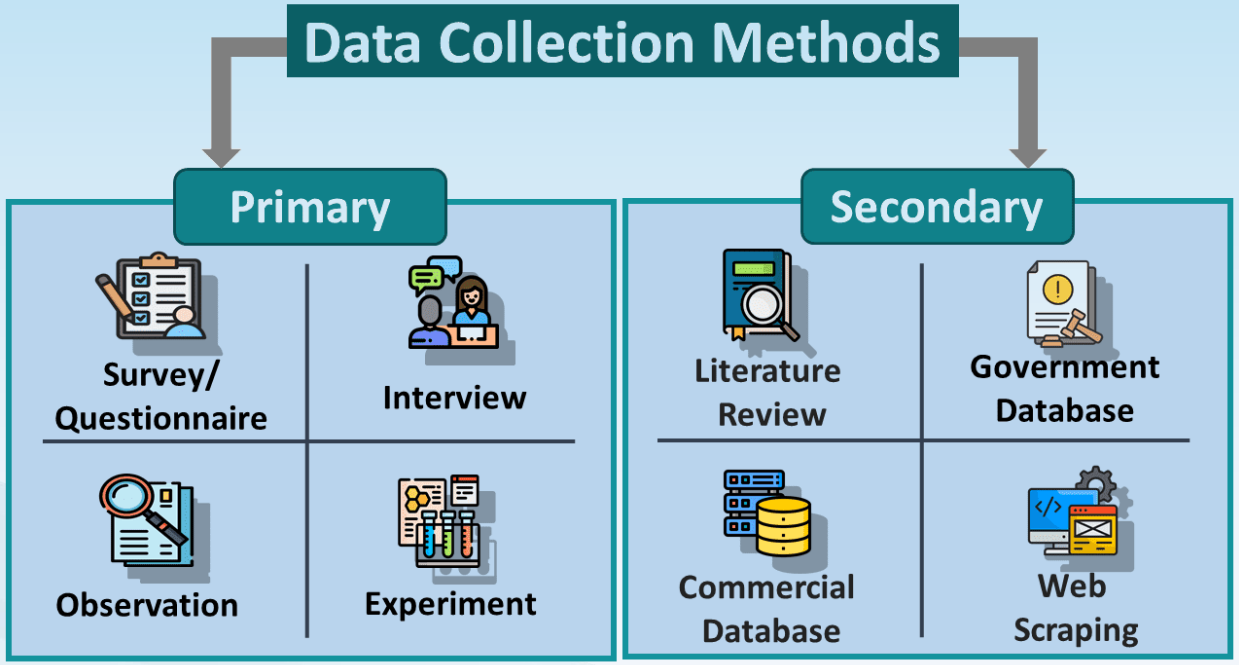
1. Nắm và hiểu được phân tích dữ liệu là gì?
2. Nắm và vận dụng được các hàm hỗ trợ thư viện Numpy

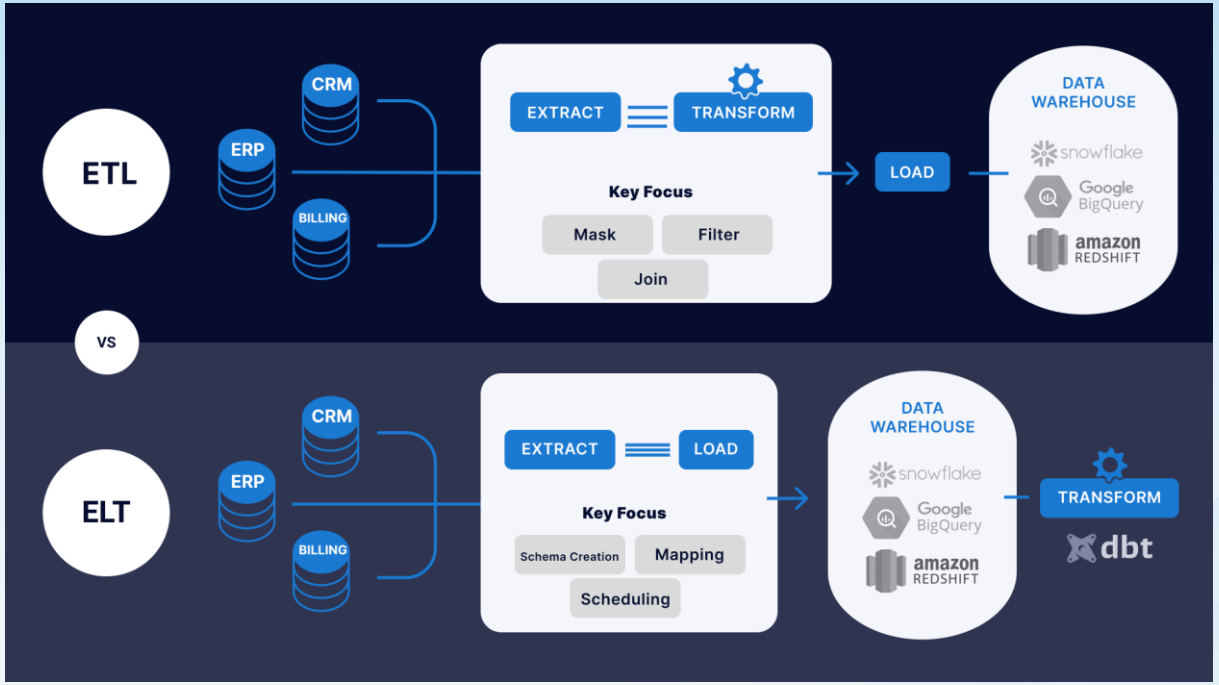
1. Giới thiệu về phân tích dữ liệu
2. Tiến trình phân tích dữ liệu
3. Phân loại phân tích dữ liệu
4. Các gói phân tích dữ liệu
5. Thư viện numpy

- Phân tích dữ liệu là quá trình quan trọng trong việc **đánh giá**, **làm sạch**, **biến đổi** và **mô hình hóa dữ liệu** nhằm mục đích tìm ra thông tin hữu ích, đưa ra kết luận và hỗ trợ ra quyết định.
- Mục đích chính của phân tích dữ liệu bao gồm việc hiểu rõ hơn về dữ liệu thông qua việc xác định các xu hướng, mẫu hình, và các đặc điểm khác giúp người phân tích có được cái nhìn sâu sắc hơn về dữ liệu, từ đó đưa ra những quyết định có cơ sở và thông minh hơn. Bên cạnh đó, phân tích dữ liệu còn có khả năng dự đoán và mô hình hóa, giúp dự báo các xu hướng và hành vi trong tương lai dựa trên dữ liệu hiện có.

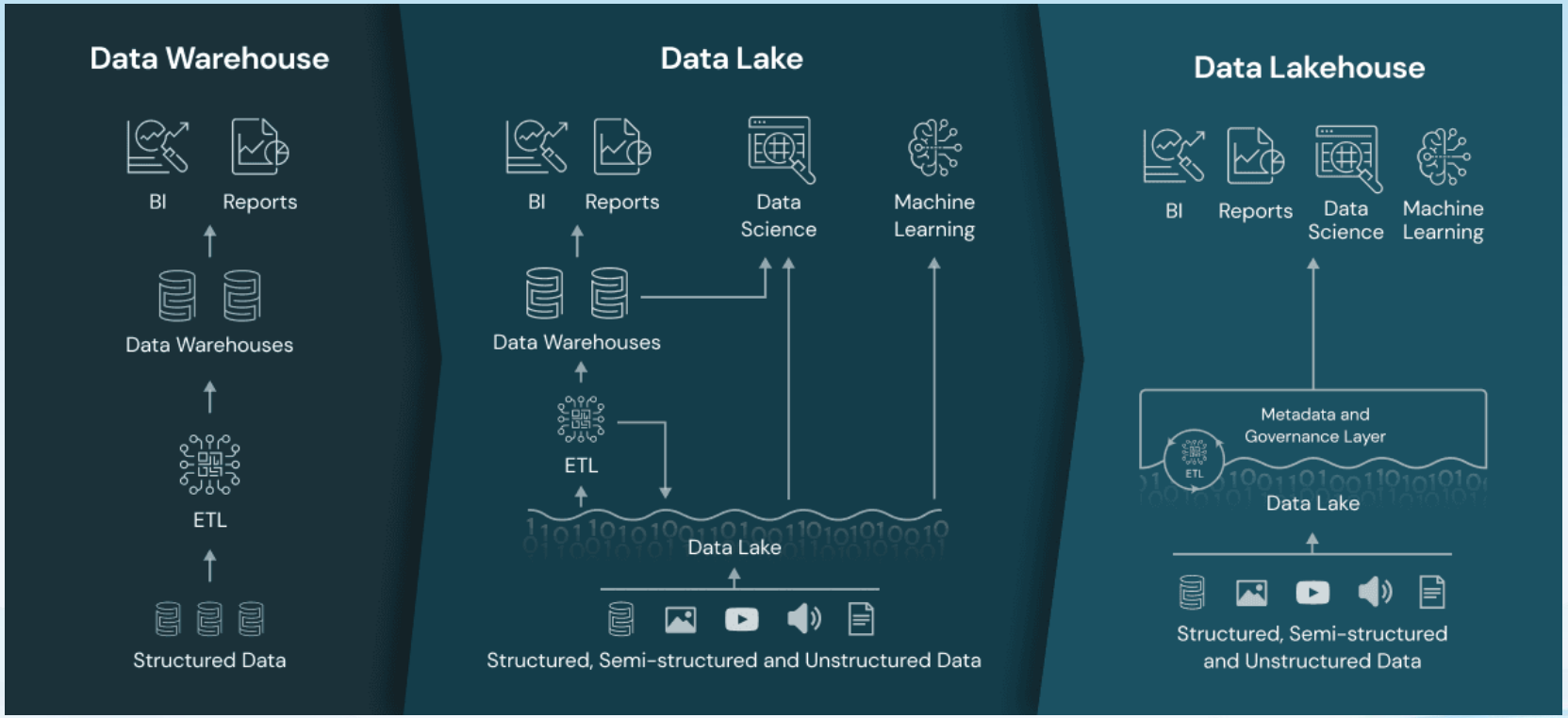
- Tiến trình phân tích dữ liệu có thể đơn giản hoặc phức tạp tùy theo mô hình đang được áp dụng vận hành gồm:
    1. Thu thập dữ liệu (Data Collection)
    2. Lưu trữ dữ liệu (Data Storage)
    3. Xử lý dữ liệu (Data Processing)
    4. Làm sạch dữ liệu (Data Cleansing)
    5. Phân tích dữ liệu (Data Analysis)
    6. Trực quan hóa dữ liệu (Data Visualization)

- Gồm các công việc xác định nguồn dữ liệu và thu thập dữ liệu từ các nguồn này
- Tuân thủ quá trình (**E**xtract, **T**ransform, **L**oad)
  - ✓ ETL: dữ liệu **trích xuất** được **chuyển đổi** thành định dạng tiêu chuẩn trước khi **tải** vào kho lưu trữ
  - ✓ ELT: dữ liệu **trích xuất** được **tải** vào kho lưu trữ, sau đó được **chuyển đổi** thành định dạng theo yêu cầu





- Dựa trên mức độ phức tạp của dữ liệu, dữ liệu có thể được lưu trữ trong data warehouse hoặc data lake. **Data warehouse** là một cơ sở dữ liệu được tối ưu để phân tích dữ liệu quan hệ từ các hệ thống giao dịch và ứng dụng kinh doanh. **Data lake** có thể lưu trữ dữ liệu có cấu trúc và phi cấu trúc mà không cần xử lý thêm do đó, cho phép thu thập mọi dữ liệu quan trọng cần thiết.

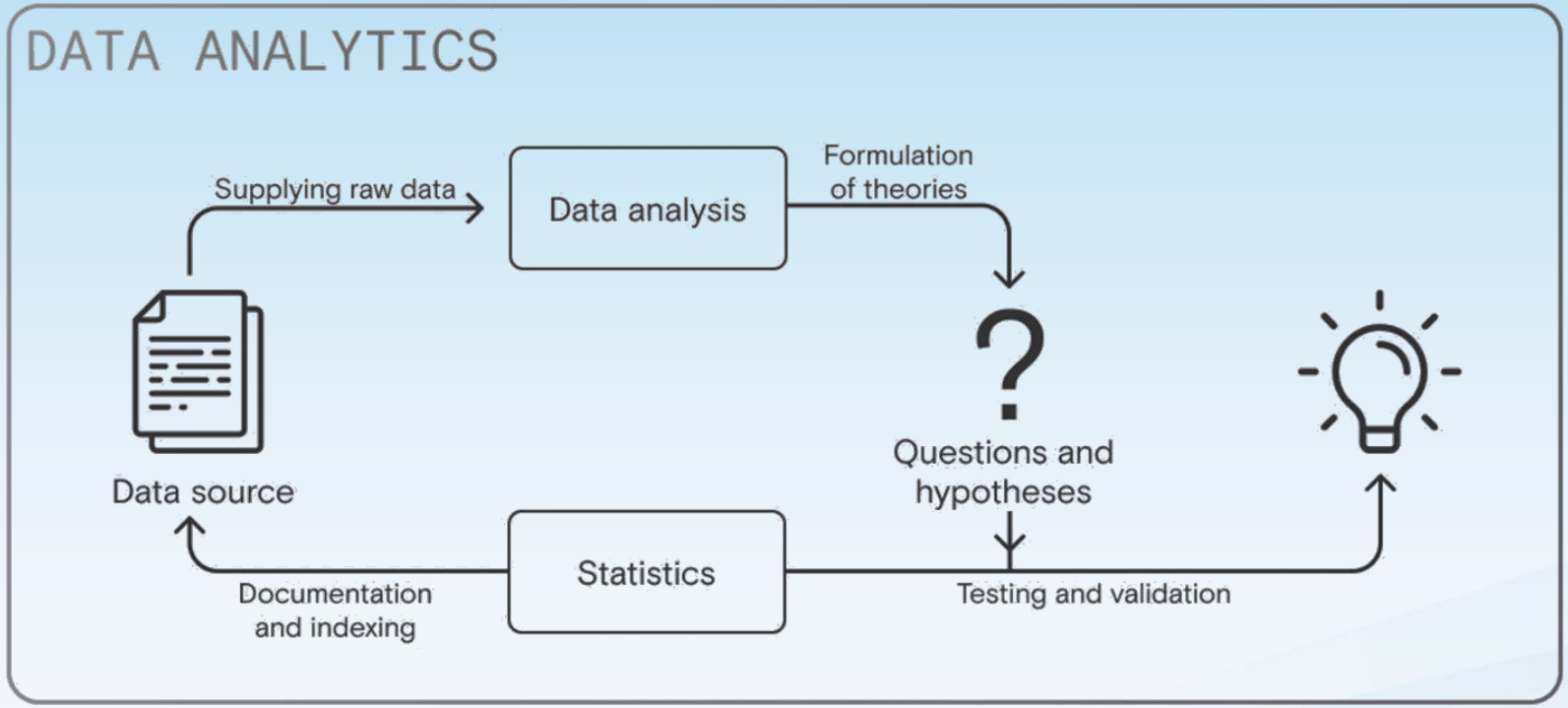


- Khi đã có dữ liệu, thì cần chuyển đổi, tổ chức để thu được kết quả chính xác từ các truy vấn phân tích
- Việc xử lý gồm một số cách thức sau:
  - ✓Xử lý tập trung: toàn bộ quá trình xử lý dữ liệu diễn ra trên một máy chủ trung tâm
  - ✓Xử lý phân tán: dữ liệu được phân tán và lưu trữ trên các máy chủ khác nhau
  - ✓Xử lý lô dữ liệu: các phần của dữ liệu tích lũy theo thời gian và được xử lý theo các lô (batch)
  - ✓Xử lý theo thời gian thực: dữ liệu được xử lý liên tục

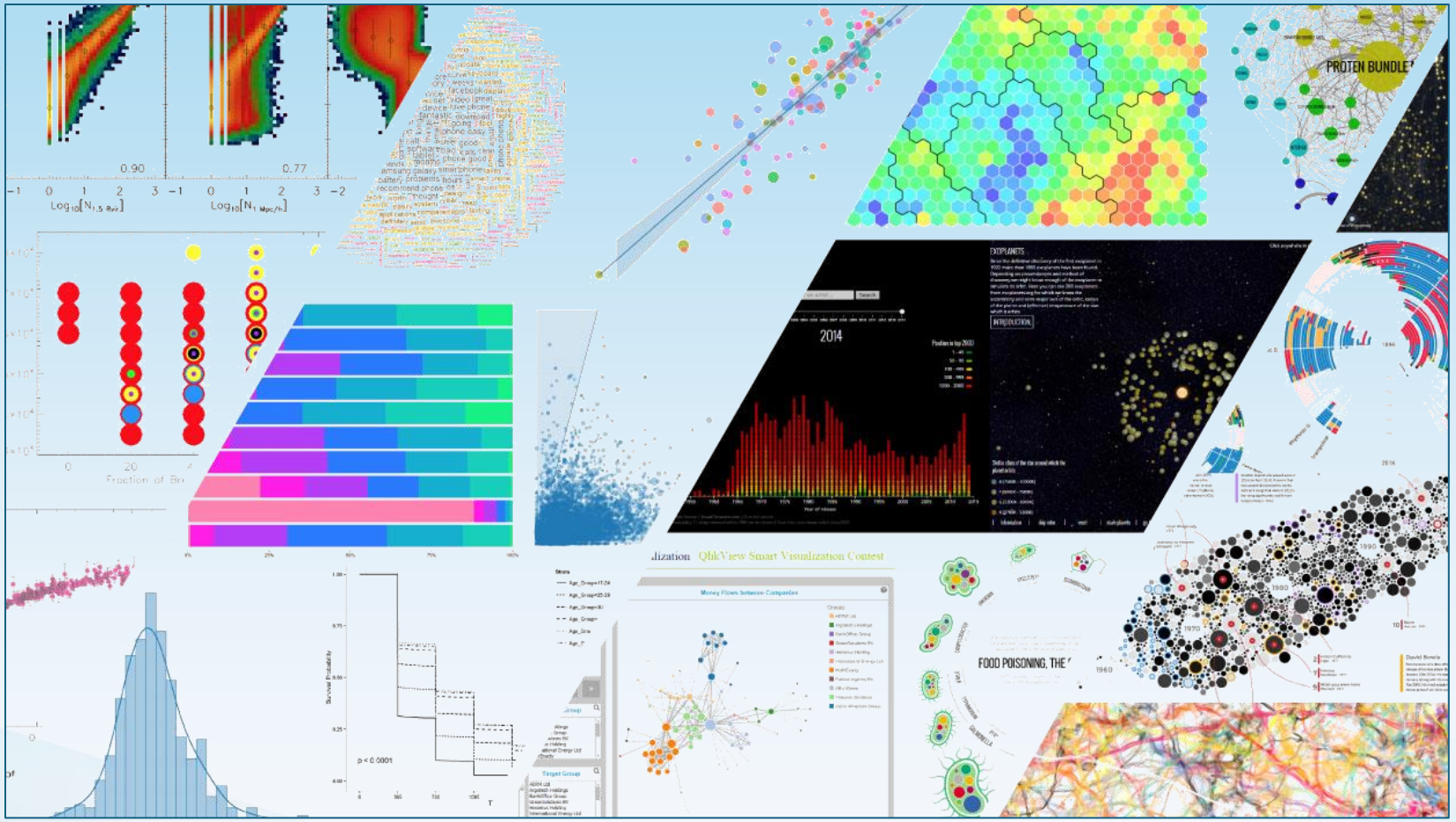
- Gồm các công việc như xóa bất kỳ các vi phạm về mặt dữ liệu như trùng lặp, không nhất quán, dư thừa hoặc định dạng sai,… cũng như lọc các dữ liệu bất kỳ bài không cần thiết (không hữu ích) trong quá trình phân tích
- Bước này thường là bắt buộc cần thực hiện trước khi phân tích dữ liệu



- Là bước chuyển đổi dữ liệu thô thành thông tin chuyên sâu hữu ích
- Tập trung vào việc sử dụng các công cụ và kỹ thuật phân tích phù hợp để trích xuất thông tin quan trọng từ các dữ liệu đã được xử lý
- Kết quả phân tích có thể làm phát sinh các yêu cầu mới để làm rõ kết quả, đòi hỏi việc thu thập thêm dữ liệu hoặc tiến hành phân tích bổ sung



- Là bước chuyển đổi dữ liệu thành dạng thông tin trực quan sinh động dễ phân tích hơn
- Các đối tượng gồm biểu đồ, đồ thị, bản đồ, hình ảnh, bảng biểu,…



- **Phân tích mô tả (Descriptive analytics):** nhằm nắm bắt những gì đã hoặc đang xảy ra trong môi trường dữ liệu thông qua việc trực quan hóa dữ liệu
- **Phân tích chuẩn đoán (Descriptive analytics):** quá trình phân tích chuyên sâu hoặc chi tiết dữ liệu nhằm nhắm được nguyên nhân xảy ra. Kỹ thuật sử dụng như đào sâu, khám phá – khai phá dữ liệu và tương quan dữ liệu
- **Phân tích dự đoán (Descriptive analytics):** sử dụng dữ liệu trong quá khứ để dự báo chính xác về các xu hướng trong tương lai. Sử dụng các kỹ thuật như máy học, mô hình dự báo, so khớp mẫu,…
- **Phân tích đề xuất (Prescriptive analytics):** không chủ dự đoán mà còn đề xuất các giải pháp tối ưu thông qua việc phân tích các tác động tìm ẩn của các lựa chọn khác nhau và đề xuất xu hướng tốt nhất. Sử dụng phân tích đồ thị, mô phỏng, mạng neural, AI và các công cụ chuyên dụng

- Phân tích cụm (Cluster analysis)
- Phân tích theo nhóm (Cohort analysis)
- Phân tích hồi quy (Regression analysis)
- Mạng nơron (Neural network)
- Phân tích nhân tố (Factor analysis)
- Khai phá dữ liệu (Data mining)
- Phân tích văn bản (Text analysis)
- Phân tích chuỗi thời gian (Time series analysis)
- Cây quyết định (Decision Trees)
- Phân tích thuộc tính (Conjoint analysis)

- Một số công cụ phân tích dữ liệu
  - ✓ PowerBI: công cụ phân tích và trực quan hóa dữ liệu
  - ✓ Python: ngôn ngữ cung cấp các thư viện có khả năng làm sạch, chuyển đổi, phân tích và trực quan dữ liệu
  - ✓ Hadoop: lưu trữ và phân tích dữ liệu
  - ✓ MongoDB: sử dụng trên các tập dữ liệu thường xuyên thay đổi
  - ✓ Talend: tích hợp và quản lý dữ liệu
  - ✓ Cassandra: cơ sở dữ liệu phân tán được sử dụng để xử lý data chunk
  - ✓ Spark: xử lý thời gian thực và phân tích dữ liệu lớn
  - ✓ STORM: hệ thống tính toán thời gian thực mã nguồn mở
  - ✓ Kafka: nền tảng phân tán trực tuyến việc lưu trữ và có khả năng chịu lỗi cao

- Trong chương trước chúng ta đã tìm hiểu về list – một cấu trúc dữ liệu cho phép lưu trữ mảng. List đơn giản và có đầy đủ chức năng cơ bản của mảng. Tuy nhiên, khi chứa mảng số với nhiều phần tử, list có tốc độ thực thi và hỗ trợ tính toán không bằng numpy arrays – một cấu trúc dữ liệu dạng mảng của thư viện numpy.
- NumPy (phát âm là /ˈnʌmpaɪ/) viết tắt cụm từ **numerical python** là một thư viện cho Python, hỗ trợ làm việc với các mảng lớn, nhiều chiều, và cung cấp các hàm toán học cấp cao hoạt động trên các mảng này
- Thường được sử dụng tính toán đại số trên ma trận & vector

- Cài đặt numpy : pip install numpy
- Khai báo thư viện : import numpy as np

Cần cài đặt thư viện trước khi khai báo, nếu không khi thực thi lệnh import sẽ báo lỗi không tìm thấy thư viện

- **Tạo mảng**

arr = np.array([value1, value2,…], dtype = <datatype_name>)

Ví dụ:

Mảng 1 chiều: arr = np.array([1, 2, 3]) # [1, 2, 3]

Mảng 2 chiều: arr = np.array([(1, 2), (3, 4)]) # [[1, 2]

[3, 4]]

Mảng 3 chiều: arr = np.array(([(), ()], [(), ()], [(), ()]))

- **Hàm dựng sẵn**
  - ✓ np.zeros(shape) # ma trận toàn phần tử **0**
  - ✓ np.ones(shape) # ma trận toàn phần tử **1**
  - ✓ np.arange(start, stop, step) # ma trận $start \leq value < stop$
  - ✓ np.full(shape, fill_value) # ma trận toàn phần tử **fill_value**
  - ✓ np.random.seed(id_memory) # giữ nguyên kết quả random
  - ✓ np.random.randn(shape) # ma trận ngẫu nhiên
- **Ma trận vuông**
  - ✓ np.eye/identity(n) # ma trận đơn vị n
  - ✓ np.diag(arr) # ma trận đường chéo hoặc trả về đường chéo



- **Đọc và lưu numpy từ file**
  - ✓ np.save/load(file, arr) # lưu/đọc ma trận vào file .npy [.npz]
  - ✓ np.savez(file, arr1, arr2,…) # lưu nhiều ma trận vào file .npz
  - ✓ np.savetxt/loadtxt(file, arr) # lưu/đọc ma trận vào file text
- **Truy xuất ma trận | array[i][j] ~ array[i, j]**

Cú pháp:

indice_start : (indice_end + 1)

-num_indices: **#** lấy số lượng num_indices ở vị trí cuối cùng

:num_indices **#** lấy một số lượng num_indices ở vị trí đầu tiên

- **Chuyển vị các chiều**
  - ✓ np.transpose(array, (d_i, d_j, …, d_n))



- **Thay đổi shape của ma trận**
  - ✓ np.reshape(new_shape) hoặc arr.reshape(new_shape) với giá trị **-1** numpy sẽ tự tính chiều còn lại
- **Ghép ma trận**
  - ✓ np.concatenate((arr1, arr2, …, arrn), axis=position)
  - ✓ np.stack/vstack/hstack/dstack([arr1, arr2, …, arrn])
- **Tách ma trận**
  - ✓ arr1, arr2 = np.split/vsplit/hsplit/dsplitk(arr)
- **Mở rộng ma trận**
  - ✓ np.expand_dims(arr, axis=position)

- **Phép toán ma trận**
  - ✓ arr.T # phép chuyển vị m x n → n x m
  - ✓ np.linalg.pinv(arr) # ma trận nghịch đảo ma trận vuông
  - ✓ np.linalg.det(arr) # định thức của ma trận
  - ✓ np.trace(arr) # tổng giá trị trên đường chéo chính
  - ✓ np.linalg.norm(arr, ord='fro') # chuẩn Frobenious
  - ✓ np.dot(arr1, arr2) | arr1.dot(arr2) | A@B # nhân hai ma trận
  - ✓ arr * value # nhân ma trận với giá trị

- **Hàm trên numpy**
  - ✓ min/max/mean/sum([axis = position])
  - ✓ minimum/maximum(arr1, arr2, …, arrn)
  - ✓ argmax/argmin(arr, axis = position) # vị trí có giá trị max/min
  - ✓ argsort(+/-arr, axis = position) # vị trí có giá trị tăng/giảm dần
  - ✓ np.exp(arr) # tính số mũ cơ số tự nhiên e, $f(x) = e^x$
- **Giữ nguyên shape**
  - ✓ np.*functionX*(…, keepdims=True) # chiều tiêu giảm = 1



1. Sử dụng phương thức thích hợp để tạo mảng numpy

```
arr = np.[…]([1, 2, 3, 4, 5])
```

[array]

2. Điền … tham số thích hợp để tạo mảng numpy 2 chiều

```
arr = np.array([1, 2, 3, 4], … =2)
```

[ndmin]

3. Điền … để kiểm tra số trường của mảng numpy

```
arr = np.array([1, 2, 3, 4])
print(arr.…)
```

[ndmin]

4. Điền … để in ra phần tử đầu tiên của mảng numpy

```
arr = np.array([1, 2, 3, 4, 5])
print(arr…)
```

[[0]]

5. Điền … để xuất ra màn hình giá trị 50 trong mảng

```
arr = np.array([10, 20, 30, 40, 50])
print(arr…)
```

[[-1]] hoặc [[4]]

6. Điền … để xuất ra màn hình giá trị 50 trong mảng

```
arr = np.array([[10, 30, 40], [50, 70, 80]])
print(arr…)
```

[[1, 0]] hoặc [[1][0]]



7. Điền … để lấy các giá trị từ thứ 2 đến thứ 4 trong mảng

```
arr = np.array([10, 15, 20, 25, 30, 35, 40])
print(arr…)
```

```
[[1:4]]
```

8. Điền … để lấy các giá trị từ thứ 2 và thứ 4 trong mảng

```
arr = np.array([10, 15, 20, 25, 30, 35, 40])
print(arr…)
```

```
[[1:4:2]]
```

9. Điền … để lấy các giá trị ở vị trí chẵn trong mảng

```
arr = np.array([10, 15, 20, 25, 30, 35, 40])
print(arr…)
```

```
[[::2]]
```

10. Điền … để lấy các giá trị ở vị trí lẻ trong mảng

```
arr = np.array([10, 15, 20, 25, 30, 35, 40])
print(arr…)
```

```
[[1::2]]
```

11. Numpy sử dụng ký tự để đại diện cho kiểu dữ liệu

i = integer	b = boolean	O = object
f = float	c = complex float	m = timedelta
M = datetime	u = unsigned integer	S = string

12. Điền … để in ra kiểu dữ liệu của mảng?

```
arr = np.array([1, 2, 3, 4])
print(arr.…)
```

```
[dtype]
```




13. Thêm tham số để chỉ định mảng có kiểu dữ liệu string

```
arr = np.array([1, 2, 3, 4], … )
```

[dtype='S']

14. Điền … để thay đổi kiểu dữ liệu sang số nguyên

```
arr = np.array([1.1, 2.1, 3.1])
newarr = arr.…
```

[astype('i')]

15. Điền … để sao chép mảng

```
arr = np.array([1, 2, 3, 4, 5])
x = arr.…
```

[copy()]

16. Điền … để tạo view của mảng

```
arr = np.array([1, 2, 3, 4, 5])
x = arr.…
```

[view()]

17. Điền … để kiểm tra kích thước (shape) của mảng

```
arr = np.array([1, 2, 3, 4, 5])
print(arr.…)
```

[shape]

18. Điền … để thay đổi kích thước mảng từ 1-D sang 2-D

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8, 9,
10, 11, 12])
newarr = arr.…(4, 3)
```

[reshape]



19. Điền … để thay đổi kích thước mảng từ 1-D sang 2-D

```
arr = np.array([[1, 2, 3, 4, 5, 6], [7, 8,
9, 10, 11, 12]])
newarr = arr.…
```

[reshape(-1)]

20. Điền … để kết 2 mảng thành 1 mảng duy nhất

```
arr1 = np.array([1, 2, 3])
arr2 = np.array([4, 5, 6])
arr = np.…((arr1, arr2))
```

[concatenate]

21. Điền … để tìm tất cả phần tử có giá trị 4

```
arr = np.array([1, 2, 3, 4, 5, 4, 4])
x = np.…(arr == 4)
```

[where]

22. Điền … để sắp xếp mảng

```
arr = np.array([3, 2, 0, 1])
x = np.…(arr)
```

[sort]

